

DataBaseViewer

Developpement d'un client de base de donnees multiplateforme en C++ / Qt

Auteur	Luka
Contexte	Preparation a un stage utilisant Qt
Langage	C++17
Framework	Qt Widgets (Qt 6)
Paradigme exige	Programmation orientee objet
Depot	github.com/bestrider14/DataBaseViewer
Date	18 juillet 2026

Table des matieres

1	Introduction et cadrage du projet	3
1.1	Contexte	3
1.2	Objectifs pedagogiques	3
1.3	Contraintes techniques	3
1.4	Methode de travail suggeree	3
1.5	Ressources	4
2	Architecture generale	5
2.1	Les classes principales	5
3	Fondations	7
3.1	Nettoyer et structurer le projet de base	7
3.2	Fenetre de connexion (UI seulement)	7
3.3	Connexion reelle a une base de donnees	8
4	Explorer et afficher	10
4.1	Lister les tables de la base connectee	10
4.2	Afficher le contenu d'une table	10
5	CRUD	12
5.1	Edition inline des cellules	12
5.2	Ajouter une nouvelle ligne	12
5.3	Supprimer une ou plusieurs lignes	13
5.4	Filtrer et trier les donnees affichees	13
6	Confort	15
6.1	Sauvegarder les profils de connexion	15
6.2	Connexions multiples / onglets	15
7	SQL avance	17
7.1	Editeur SQL brut	17
7.2	Tunnel SSH pour connexion a une base distante	17
8	Stretch goals	19
8.1	Query builder graphique	19
8.2	Polish : theme, icones, robustesse	19

1 Introduction et cadrage du projet

1.1 Contexte

Ce document sert de guide pour la réalisation de **DataBaseViewer**, un projet personnel dont le but est de se remettre à niveau en C++ et en Qt avant un stage. L'application visée est un client de base de données dans l'esprit de pgAdmin : se connecter à une base (locale ou distante, potentiellement via un tunnel SSH vers un conteneur Docker), explorer les tables, et effectuer des opérations CRUD (Create, Read, Update, Delete) via une interface graphique. Des objectifs plus avancés (éditeur SQL libre, constructeur de requêtes graphique) sont prévus en fin de parcours.

Le document est structuré comme un travail pratique universitaire : chaque partie correspond à une **issue** du dépôt GitHub, avec un objectif, des concepts Qt à explorer, une piste d'architecture orientée objet, un exemple de code illustratif quand c'est pertinent, et des critères d'acceptation. **Les exemples de code sont volontairement partiels** (souvent uniquement des déclarations de classes ou de courts extraits) : le but est de illustrer un mécanisme ou un patron de conception, pas de fournir une solution complète. L'implémentation et les choix de détail restent à faire par toi-même.

1.2 Objectifs pédagogiques

- Se familiariser avec la syntaxe C++ moderne (C++17) après une pause.
- Pratiquer le modèle signaux/slots et l'architecture événementielle de Qt.
- Maîtriser le patron **Model/View** de Qt (`QSqlTableModel`, `QSqlQueryModel`, `QTableView`).
- Concevoir une architecture orientée objet propre : séparation des responsabilités, encapsulation, faible couplage entre l'UI et l'accès aux données.
- Manipuler des ressources qui doivent être correctement libérées (connexions DB, processus externes) – un bon terrain d'entraînement pour RAII et la gestion du cycle de vie des objets Qt (parentage `QObject`).

1.3 Contraintes techniques

CRITERES D'ACCEPTATION

- Le code doit rester **orienté objet** : pas de logique métier qui s'accumule directement dans `MainWindow` ou dans des fonctions libres. Chaque responsabilité (connexion DB, tunnel SSH, gestion des profils, etc.) doit vivre dans sa propre classe.
- C++17, modules Qt Widgets + Qt SQL (`QT += widgets sql`).
- Une classe = un fichier `.h/.cpp` (convention Qt standard).
- Pas de mot de passe stocké en clair sans réflexion explicite sur la question (voir section correspondante).
- Chaque connexion ouverte doit pouvoir être fermée proprement (pas de warning Qt au runtime, pas de processus fantôme pour le tunnel SSH).

1.4 Méthode de travail suggérée

1. Une issue GitHub = une unité de travail. Termine et teste manuellement une issue avant de passer à la suivante – l'ordre des phases n'est pas arbitraire, chaque phase s'appuie sur la précédente.

2. Commit regulierement, avec des messages qui expliquent le **pourquoi** plutot que le **quoi**.
3. Avant de coder une classe, écris d'abord son interface publique (le **.h**) : quelles methodes, quels signaux, quelles responsabilites. Cela force a clarifier la conception avant de plonger dans l'implementation.
4. Quand un exemple de code est fourni dans ce document, considere-le comme un point de depart a adapter, pas comme du code a copier-coller tel quel.

1.5 Ressources

- Documentation officielle Qt : doc.qt.io – utilise la barre de recherche pour trouver la reference de n'importe quelle classe (ex. chercher « QSqlTableModel »).
- Le module Qt SQL dans son ensemble est documente sous l'entree « Qt SQL » de la documentation.
- Qt Creator integre l'aide contextuelle (F1 sur un symbole) qui pointe directement vers la documentation de la classe.

2 Architecture generale

Avant de plonger dans les issues, voici une proposition de decoupage en classes qui respecte la contrainte de programmation orientee objet. Ce n'est pas une obligation absolue, mais un squelette raisonnable qui separe clairement les responsabilites : **acces aux donnees**, **presentation**, et **orchestration**.

```
MainWindow
|-- QTabWidget
|   |-- DatabaseSessionWidget      (un par connexion ouverte)
|   |   |-- DatabaseConnection    (+ SshTunnel optionnel)
|   |   |-- TableExplorerWidget  (liste des tables)
|   |   |-- TableDataWidget      (QTableView + modele SQL)
|   |   |-- SqlConsoleWidget     (editeur SQL libre)
|-- ConnectionDialog              (modal, ponctuel)
|-- ConnectionProfileStore        (persistance via QSettings)
```

2.1 Les classes principales

Classe	Responsabilite
ConnectionInfo	Structure de valeur : parametres necessaires pour ouvrir une connexion (moteur, host, port, identifiants, options SSH).
DatabaseConnection	Encapsule un <code>QSqlDatabase</code> (et un <code>SshTunnel</code> optionnel). Expose <code>open()</code> , <code>close()</code> , <code>tableNames()</code> , et des signaux d'etat. C'est la seule classe qui parle directement a <code>QSqlDatabase</code> .
SshTunnel	Gere le cycle de vie d'un tunnel SSH (ouverture, fermeture propre) pour les connexions distantes.
ConnectionProfileStore	Sauvegarde/charge des profils de connexion nommes via <code>QSettings</code> .
ConnectionDialog	Boite de dialogue qui collecte les champs et produit un <code>ConnectionInfo</code> .
TableExplorerWidget	Affiche la liste des tables d'une connexion, emet un signal quand l'utilisateur en selectionne une.
TableDataWidget	Affiche et permet d'editer le contenu d'une table (ajout/suppression/tri/filtre).
SqlConsoleWidget	Editeur de requetes SQL libres avec affichage des resultats.
DatabaseSessionWidget	Compose les trois widgets ci-dessus pour une connexion donnee ; correspond a un onglet.
MainWindow	Orchestration generale : gere les onglets, ouvre les dialogues, ne contient pas de logique metier.

ARCHITECTURE ORIENTEE OBJET SUGGEREE

L'idee centrale : **aucune classe d'interface ne doit manipuler `QSqlDatabase` directement**, sauf `DatabaseConnection`. Cela veut dire que `TableDataWidget` ou `SqlConsoleWidget` recoivent une reference/pointeur vers une `DatabaseConnection` (ou directement vers un `QSqlDatabase` deja ouvert), mais

ne savent pas comment une connexion s'ouvre, se ferme, ou passe par un tunnel SSH. Ce découplage te permettra d'ajouter le tunnel SSH plus tard (Phase 5) sans toucher au code d'affichage des tables.

Deux exemples de squelettes de classes pour donner le ton (declarations uniquement, a completer et adapter) :

```
// connectioninfo.h
enum class DbEngine { SQLite, PostgreSQL, MySQL };

struct ConnectionInfo
{
    DbEngine engine = DbEngine::PostgreSQL;
    QString host;
    int port = 5432;
    QString databaseName;
    QString username;
    QString password;
    bool useSshTunnel = false;
    // ... parametres du tunnel SSH a ajouter en Phase 5
};
```

```
// databaseconnection.h
class DatabaseConnection : public QObject
{
    Q_OBJECT
public:
    explicit DatabaseConnection(QObject *parent = nullptr);
    ~DatabaseConnection() override;

    bool open(const ConnectionInfo &info);
    void close();
    bool isOpen() const;

    QStringList tableNames() const;
    QSqlDatabase database() const { return m_db; }
    QString lastError() const;

signals:
    void connected();
    void disconnected();
    void errorOccurred(const QString &message);

private:
    QSqlDatabase m_db;
    QString m_connectionName;
};
```

3 Fondations

3.1 Nettoyer et structurer le projet de base

Issue *GitHub 1*

OBJECTIF

Partir du squelette Qt Creator par défaut et le transformer en une base saine pour la suite du projet.

CONCEPTS A EXPLORER

- Organisation d'un projet Qt Widgets (`.pro`, separation `.h/.cpp/.ui`).
- Ajout du module SQL : `QT += widgets sql`.
- Organisation en dossiers si tu veux dépasser le flat layout (`src/`, `ui/`).

CRITERES D'ACCEPTATION

- [] Le projet compile toujours apres reorganisation.
- [] Le module `sql` est ajoute au `.pro`.
- [] Un README decrit le projet et comment le compiler.
- [] `.gitignore` couvre correctement les dossiers de build.

3.2 Fenetre de connexion (UI seulement)

Issue *GitHub 2*

OBJECTIF

Creer une boite de dialogue (`QDialog`) permettant de saisir les parametres de connexion : moteur, host, port, nom de la base, utilisateur, mot de passe.

CONCEPTS A EXPLORER

- `QDialog`, layouts (`QFormLayout`).
- Qt Designer (`.ui`) vs construction du formulaire en code.
- `QComboBox` pour choisir le driver, `QLineEdit` avec `EchoMode::Password`.

ARCHITECTURE ORIENTEE OBJET SUGGEREE

C'est ici que naît la classe `ConnectionDialog`. Sa seule responsabilite est de collecter des donnees et de produire un `ConnectionInfo` – elle ne doit pas savoir comment on ouvre une connexion.

```
// connectiondialog.h
class ConnectionDialog : public QDialog
{
    Q_OBJECT
public:
    explicit ConnectionDialog(QWidget *parent = nullptr);
    ConnectionInfo connectionInfo() const;

private slots:
    void onEngineChanged(int index);
    void updateAcceptButtonState();

private:
    QComboBox *m_engineCombo;
    QLineEdit *m_hostEdit;
    QLineEdit *m_portEdit;
    QLineEdit *m_databaseEdit;
    QLineEdit *m_userEdit;
    QLineEdit *m_passwordEdit;
};
```

CRITERES D'ACCEPTATION

- [] Une fenetre modale s'ouvre au demarrage (ou via un menu « Nouvelle connexion »).
- [] Les champs s'adaptent selon le moteur choisi (SQLite n'a pas besoin de host/port).
- [] Le bouton « Connecter » est desactive tant que les champs requis ne sont pas remplis.
- [] Pas encore de vraie connexion – uniquement l'UI et la collecte des valeurs.

3.3 Connexion réelle a une base de donnees

Issue *GitHub 3*

OBJECTIF

Utiliser le `ConnectionInfo` produit par le dialogue pour ouvrir une vraie connexion via `QSqlDatabase`, et gerer les erreurs proprement.

CONCEPTS A EXPLORER

- `QSqlDatabase::addDatabase(driver, connectionName)`, `setHostName/setPort/setDatabaseName/setUserName/setPassword`.
- `QSqlDatabase::open()` et `lastError()`.
- Drivers Qt SQL disponibles sur ta machine (`QSQLITE`, `QPSQL`, `QMYSQL`).

ARCHITECTURE ORIENTEE OBJET SUGGEREE

C'est le coeur de la classe `DatabaseConnection` presentee plus haut. Remarque le parametre `connectionName` : Qt identifie chaque connexion par un nom unique (chaîne de caracteres), ce qui devient important des que tu veux plusieurs connexions simultanees (Phase 4). Genere un identifiant unique par instance, par exemple avec `QUuid::createUuid()`.


```
// databaseconnection.cpp (extrait, a completer)
bool DatabaseConnection::open(const ConnectionInfo &info)
{
    m_connectionName = QUuid::createUuid().toString();
    m_db = QSqlDatabase::addDatabase(driverNameFor(info.engine), m_connectionName);

    // A toi de completer : setHostName, setPort, setDatabaseName, ...

    if (!m_db.open()) {
        emit errorOccurred(m_db.lastError().text());
        return false;
    }
    emit connected();
    return true;
}
```

PIEGE / POINT D'ATTENTION

Les drivers Qt SQL (**QPSQL**, **QMYSQL**) ne sont pas toujours inclus par default selon ton installation Qt – attends-toi a devoir chercher comment les activer ou les installer. Verifie avec **QSqlDatabase::drivers()** quels drivers sont disponibles chez toi.

CRITERES D'ACCEPTATION

- ☐ Connexion reussie a une base SQLite locale (fichier de test).
- ☐ Connexion reussie a une base PostgreSQL locale (via Docker par exemple).
- ☐ Message d'erreur clair si la connexion echoue.
- ☐ Apres connexion reussie, la fenetre de connexion se ferme et la fenetre principale s'affiche.

4 Explorer et afficher

4.1 Lister les tables de la base connectee

Issue *GitHub* 4

OBJECTIF

Afficher dans un panneau lateral la liste des tables disponibles dans la base connectee.

CONCEPTS A EXPLORER

- `QSqlDatabase::tables()`.
- `QListView` (ou `QTreeView`) + `QStringListModel`.
- `QSplitter` pour separer la sidebar de la zone principale.

ARCHITECTURE ORIENTEE OBJET SUGGEREE

`TableExplorerWidget` encapsule la liste et expose un signal `tableSelected(const QString &tableName)`. Le widget parent (`DatabaseSessionWidget`) connecte ce signal au slot qui charge les donnees – ni l'un ni l'autre n'a besoin de connaitre les details internes de son voisin.

```
// tableexplorerwidget.h
class TableExplorerWidget : public QWidget
{
    Q_OBJECT
public:
    explicit TableExplorerWidget(QWidget *parent = nullptr);
    void setTables(const QStringList &tableNames);

signals:
    void tableSelected(const QString &tableName);

private:
    QListView *m_listView;
    QStringListModel *m_model;
};
```

CRITERES D'ACCEPTATION

- [] La sidebar affiche toutes les tables de la base connectee.
- [] Cliquer sur une table emet un signal avec le nom de la table.
- [] La liste se rafraichit si on se reconnecte a une autre base.

4.2 Afficher le contenu d'une table

Issue *GitHub* 5

OBJECTIF

Quand une table est selectionnee dans la sidebar, afficher son contenu dans une grille.

CONCEPTS A EXPLORER

- `QSqlTableModel` vs `QSqlQueryModel` – comprendre la difference et quand choisir l’un ou l’autre.
- `QTableView::setModel`, `QSqlTableModel::setTable` + `select()`.
- `headerData` pour l’affichage des noms de colonnes.

```
// tabledatawidget.cpp (extrait)
void TableDataWidget::showTable(const QString &tableName)
{
    auto *model = new QSqlTableModel(this, m_connection->database());
    model->setTable(tableName);
    model->select();
    m_tableView->setModel(model);
}
```

PIEGE / POINT D'ATTENTION

Si tu recrees un `QSqlTableModel` a chaque changement de table sans detruire l’ancien (ou sans le reparenter correctement), tu risques des fuites. Le parametre `this` comme parent Qt s’occupe de la destruction automatique – mais reflechis a ce qui se passe si `showTable` est appelee plusieurs fois de suite sur le meme widget.

CRITERES D'ACCEPTATION

- [] Selectionner une table affiche ses donnees dans un `QTableView`.
- [] Les noms de colonnes sont visibles et lisibles.
- [] Changer de table met a jour correctement la vue (pas de donnees fantomes).
- [] Gestion basique des tables vides ou tres grandes (au moins ne pas planter).

5 CRUD

5.1 Edition inline des cellules

Issue *GitHub* 6

OBJECTIF

Permettre de modifier une valeur directement dans la grille et de la persister en base.

CONCEPTS A EXPLORER

- `QSqlTableModel::EditStrategy` : `OnFieldChange`, `OnRowChange`, `OnManualSubmit` – ce sont trois compromis differents entre reactivite et controle.
- `submitAll()` / `revertAll()`.

```
model->setEditStrategy(QSqlTableModel::OnManualSubmit);
// ... l'utilisateur edite une ou plusieurs cellules ...
if (!model->submitAll()) {
    QMessageBox::warning(this, tr("Erreur"), model->lastError().text());
    model->revertAll();
}
```

CRITERES D'ACCEPTATION

- [] Double-clic sur une cellule permet de modifier sa valeur.
- [] La modification est persistee en base apres validation.
- [] Une erreur de contrainte est affichee sans crash.
- [] Tu as choisi une `EditStrategy` en connaissance de cause (documente pourquoi).

5.2 Ajouter une nouvelle ligne

Issue *GitHub* 7

OBJECTIF

Permettre d'insérer une nouvelle ligne vide dans la table affichee.

CONCEPTS A EXPLORER

- `QSqlTableModel::insertRow()` / `insertRecord()`.
- Gestion des colonnes auto-incrementees / cles primaires.

CRITERES D'ACCEPTATION

- [] Un bouton permet d'ajouter une ligne vide.
- [] La ligne peut etre remplie puis sauvegardee.
- [] Une ligne ajoutee puis annulee ne pollue pas la base.

5.3 Supprimer une ou plusieurs lignes

Issue GitHub 8

OBJECTIF

Permettre de selectionner une ou plusieurs lignes et de les supprimer, avec confirmation.

CONCEPTS A EXPLORER

- `QTableView::selectionModel()`, selection multiple.
- `QSqlTableModel::removeRow()` / `removeRows()`.
- `QMessageBox::question` pour confirmer une action destructive.

CRITERES D'ACCEPTATION

- [] La suppression demande confirmation.
- [] La suppression est repercutee en base.
- [] Annuler la confirmation ne supprime rien.

5.4 Filtrer et trier les donnees affichees

Issue GitHub 9

OBJECTIF

Ajouter une barre de recherche/filtre et permettre le tri par colonne.

CONCEPTS A EXPLORER

- `QSqlTableModel::setFilter()` (filtre SQL cote serveur) vs `QSortFilterProxyModel` (filtre cote client).
- `QTableView::setSortingEnabled(true)`.

```
model->setFilter(QString("nom LIKE '%1%'").arg(searchText));
model->select();
```

PIEGE / POINT D'ATTENTION

Construire une clause **WHERE** par concatenation directe de texte utilisateur pose un risque d'injection SQL des que la valeur contient des guillemets ou des caracteres speciaux. Renseigne-toi sur comment echapper correctement la valeur, ou envisage `QSortFilterProxyModel` qui evite le probleme en filtrant cote client.

CRITERES D'ACCEPTATION

- [] Un champ de recherche filtre les lignes affichees.
- [] Cliquer sur un en-tete de colonne trie les donnees.
- [] Le filtre/tri fonctionne correctement combine a l'edition.

6 Confort

6.1 Sauvegarder les profils de connexion

Issue *GitHub* 10

OBJECTIF

Ne plus avoir à retaper les infos de connexion à chaque lancement.

CONCEPTS A EXPLORER

- `QSettings` (stockage cle/valeur multiplateforme).
- Ne jamais stocker un mot de passe en clair sans y avoir réfléchi.

ARCHITECTURE ORIENTEE OBJET SUGGEREE

`ConnectionProfileStore` encapsule toute la logique `QSettings`. Le reste de l'application manipule des `ConnectionInfo` nommés, sans savoir où ni comment ils sont persistés.

```
// connectionprofilestore.h
class ConnectionProfileStore
{
public:
    void save(const QString &profileName, const ConnectionInfo &info);
    ConnectionInfo load(const QString &profileName) const;
    QStringList profileNames() const;
    void remove(const QString &profileName);
};
```

PIEGE / POINT D'ATTENTION

Réfléchis avant de sauvegarder `info.password` tel quel dans `QSettings` – c'est un piège classique. Options possibles : ne pas le sauvegarder du tout (le redemander à chaque connexion), ou explorer un stockage sécurisé du système d'exploitation (ex. `QtKeychain`).

CRITERES D'ACCEPTATION

- [] Une connexion peut être nommée et sauvegardée.
- [] La liste des profils sauvegardés apparaît au démarrage.
- [] Un profil peut être supprimé.

6.2 Connexions multiples / onglets

Issue *GitHub* 11

OBJECTIF

Pouvoir avoir plusieurs bases de données ouvertes en même temps, comme les onglets d'un navigateur.

CONCEPTS A EXPLORER

- `QTabWidget`.
- Unicité des noms de connexion `QSqlDatabase` entre onglets.
- `QSqlDatabase::removeDatabase()` pour une fermeture propre.

ARCHITECTURE ORIENTEE OBJET SUGGEREE

C'est le moment où `DatabaseSessionWidget` prend tout son sens : chaque onglet possède sa propre instance (sa propre `DatabaseConnection`, son propre `TableExplorerWidget`, etc.), et `MainWindow` se contente d'ajouter/retirer des `DatabaseSessionWidget` dans un `QTabWidget`, sans connaître leurs détails internes.

CRITERES D'ACCEPTATION

- [] Ouvrir une nouvelle connexion crée un nouvel onglet indépendant.
- [] Fermer un onglet ferme proprement la connexion associée.
- [] Les onglets n'interfèrent pas entre eux.

7 SQL avance

7.1 Editeur SQL brut

Issue *GitHub 12*

OBJECTIF

Ajouter un panneau où l'utilisateur peut écrire et exécuter une requête SQL arbitraire, avec affichage des résultats.

CONCEPTS A EXPLORER

- `QPlainTextEdit` pour la saisie.
- `QSqlQuery` pour exécuter du SQL arbitraire.
- `QSqlQueryModel` pour afficher le résultat d'un `SELECT`.
- Distinguer les requêtes qui retournent des lignes de celles qui n'en retournent pas.

```
// sqlconsolewidget.cpp (extrait)
void SqlConsoleWidget::executeQuery(const QString &sql)
{
    QSqlQuery query(m_connection->database());
    if (!query.exec(sql)) {
        emit errorOccurred(query.lastError().text());
        return;
    }
    if (query.isSelect()) {
        auto *resultModel = new QSqlQueryModel(this);
        resultModel->setQuery(std::move(query));
        m_resultView->setModel(resultModel);
    }
}
```

CRITERES D'ACCEPTATION

- [] L'utilisateur peut taper une requête SQL et l'exécuter.
- [] Les résultats d'un `SELECT` s'affichent dans une grille.
- [] Une requête invalide affiche une erreur claire sans crash.

7.2 Tunnel SSH pour connexion a une base distante

Issue *GitHub 13*

OBJECTIF

Se connecter à une base qui tourne dans un conteneur Docker sur un serveur distant, en passant par un tunnel SSH.

CONCEPTS A EXPLORER

- Principe du port forwarding local (`ssh -L port_local:localhost:port_db user@host`).
- Deux approches a comparer : piloter le client `ssh` systeme via `QProcess`, ou utiliser une bibliotheque comme `libssh2/libssh` directement en C++.
- Cycle de vie du tunnel : ouverture avant la connexion DB, fermeture propre a la deconnexion.

ARCHITECTURE ORIENTEE OBJET SUGGEREE

`SshTunnel` est une classe a part entiere, independante de `DatabaseConnection`. `DatabaseConnection::open()` peut, si `info.useSshTunnel` est vrai, demarrer un `SshTunnel` avant d'ouvrir la connexion SQL sur `localhost` au port local du tunnel. `TableDataWidget` et `SqlConsoleWidget` n'ont aucune idee qu'un tunnel existe – c'est exactement le benefice du decouplage propose en debut de document.

```
// sshtunnel.h (approche QProcess, la plus simple pour commencer)
class SshTunnel : public QObject
{
    Q_OBJECT
public:
    explicit SshTunnel(QObject *parent = nullptr);
    bool start(const QString &sshHost, int localPort, int remotePort);
    void stop();
    bool isActive() const;

signals:
    void tunnelReady();
    void tunnelFailed(const QString &message);

private:
    QProcess *m_process = nullptr;
};
```

PIEGE / POINT D'ATTENTION

Commence par l'approche `QProcess` + client `ssh` systeme, nettement plus simple pour un premier essai qu'une bibliotheque comme `libssh`. Tu pourras explorer une solution plus « pure C++ » plus tard si tu veux aller plus loin.

CRITERES D'ACCEPTATION

- [] Le dialogue de connexion permet d'activer « via tunnel SSH ».
- [] La connexion a la base Docker distante fonctionne via le tunnel.
- [] Le tunnel se ferme proprement quand la connexion DB est fermee.

8 Stretch goals

Cette dernière phase est optionnelle : à faire uniquement si le temps le permet avant le stage. Elle n'est pas bloquante pour le reste du projet.

8.1 Query builder graphique

OBJECTIF

Permettre de construire une requête SQL (au moins un **SELECT** avec colonnes/**WHERE/ORDER BY**) via une interface graphique, sans écrire de SQL à la main.

CONCEPTS À EXPLORER

- Modéliser une requête en objets C++ (colonnes, conditions, tri) avant de la traduire en texte SQL.
- UI dynamique : ajouter/retirer des conditions de filtre à la volée.

CRITERES D'ACCEPTATION

- [] L'utilisateur peut choisir une table, des colonnes, et des conditions **WHERE**.
- [] Le SQL généré est visible avant exécution.

8.2 Polish : theme, icones, robustesse

OBJECTIF

Rendre l'application plus agréable à utiliser et plus robuste, une fois les fonctionnalités de base en place.

CONCEPTS À EXPLORER

- Icones pour les actions, feedback visuel pendant les opérations longues.
- Fermeture propre de toutes les connexions ouvertes à la fermeture de l'application.
- Tests unitaires sur la logique non-UI (**QTest**) si tu veux pousser plus loin.

CRITERES D'ACCEPTATION

- [] Au moins trois améliorations de la liste ci-dessus sont implémentées.
- [] L'application ne laisse pas de connexions DB ouvertes en quittant.